

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 2

Roll no: 54 PRN No: 12414148 Name of Student: Sanika Kulkarni

Subject Name: Deep Learning

Problem Statement:

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix

Algorithm:

1. Start the program.
2. Import the required libraries.
3. Load the Iris or Wine dataset.
4. Split the dataset into training and testing sets.
5. Standardize the feature values.
6. Create the MLP classifier.
7. Train the model using the training data.
8. Predict the class labels for the test data.
9. Evaluate the model using performance metrics.
10. Display the Confusion Matrix, Loss Curve, and Actual vs Predicted graph.
11. Repeat the process for another train-test split scenario.
12. Compare the performance of both scenarios.
13. Stop the program.

Code:

Scenario 1 (80% Training – 20% Testing): The MLP classifier is trained using 80% of the Iris dataset and evaluated on the remaining 20% to measure its performance.

Scenario 2 (90% Training – 10% Testing): The same MLP classifier is trained using 90% of the Iris dataset and tested on the remaining 10% to observe the effect of using more training data.

Comparison: The performance of both scenarios is compared using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and graphical visualizations to analyze the impact of different train-test split ratios.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report,
precision_score, recall_score, f1_score, ConfusionMatrixDisplay

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Target"] = iris.target
print("Iris Dataset:")
print(df.head())
print("\nTarget Classes:")
print(iris.target_names)
X = iris.data
y = iris.target

print("\nSCENARIO 1 (80% Train - 20% Test)")
X_train1, X_test1, y_train1, y_test1 = train_test_split(X, y, test_size=0.2, random_state=42)
scaler1 = StandardScaler()
X_train1 = scaler1.fit_transform(X_train1)
X_test1 = scaler1.transform(X_test1)
mlp1 = MLPClassifier(hidden_layer_sizes=(10,10), activation='relu', solver='adam',
max_iter=1000, random_state=42)
mlp1.fit(X_train1, y_train1)
y_pred1 = mlp1.predict(X_test1)
accuracy1 = accuracy_score(y_test1, y_pred1)
precision1 = precision_score(y_test1, y_pred1, average='weighted')
recall1 = recall_score(y_test1, y_pred1, average='weighted')
f11 = f1_score(y_test1, y_pred1, average='weighted')

print("Accuracy :", accuracy1)
print("Precision:", precision1)
print("Recall  :", recall1)
print("F1 Score :", f11)
```

```

print("\nConfusion Matrix")
print(confusion_matrix(y_test1, y_pred1))
print("\nClassification Report")
print(classification_report(y_test1, y_pred1))
fig, axes = plt.subplots(1, 2, figsize=(12,5))
ConfusionMatrixDisplay.from_predictions(y_test1, y_pred1, cmap="Greens", ax=axes[0])
axes[0].set_title("Scenario 1 Confusion Matrix")
axes[1].plot(mlp1.loss_curve_, marker='o')
axes[1].set_title("Scenario 1 Loss Curve")
axes[1].set_xlabel("Iterations")
axes[1].set_ylabel("Loss")
axes[1].grid(True)
plt.tight_layout()
plt.show()

plt.figure(figsize=(8,4))
plt.scatter(range(len(y_test1)), y_test1, marker='o', s=70, label='Actual')
plt.scatter(range(len(y_pred1)), y_pred1, marker='x', s=70, label='Predicted')
plt.xlabel("Sample Number")
plt.ylabel("Class")
plt.title("Scenario 1: Actual vs Predicted")
plt.yticks([0,1,2], iris.target_names)
plt.legend()
plt.grid(True)
plt.show()

print("\nSCENARIO 2 (90% Train - 10% Test)")
X_train2, X_test2, y_train2, y_test2 = train_test_split(X, y, test_size=0.1, random_state=42)
scaler2 = StandardScaler()
X_train2 = scaler2.fit_transform(X_train2)
X_test2 = scaler2.transform(X_test2)
mlp2 = MLPClassifier(hidden_layer_sizes=(10,10), activation='relu', solver='adam',
max_iter=1000, random_state=42)
mlp2.fit(X_train2, y_train2)
y_pred2 = mlp2.predict(X_test2)
accuracy2 = accuracy_score(y_test2, y_pred2)
precision2 = precision_score(y_test2, y_pred2, average='weighted')
recall2 = recall_score(y_test2, y_pred2, average='weighted')
f12 = f1_score(y_test2, y_pred2, average='weighted')

print("Accuracy :", accuracy2)
print("Precision:", precision2)
print("Recall  :", recall2)
print("F1 Score :", f12)
print("\nConfusion Matrix")
print(confusion_matrix(y_test2, y_pred2))

```

```

print("\nClassification Report")
print(classification_report(y_test2, y_pred2))
fig, axes = plt.subplots(1, 2, figsize=(12,5))
ConfusionMatrixDisplay.from_predictions(y_test2, y_pred2, cmap="Reds", ax=axes[0])
axes[0].set_title("Scenario 2 Confusion Matrix")
axes[1].plot(mlp2.loss_curve_, marker='o')
axes[1].set_title("Scenario 2 Loss Curve")
axes[1].set_xlabel("Iterations")
axes[1].set_ylabel("Loss")
axes[1].grid(True)
plt.tight_layout()
plt.show()
plt.figure(figsize=(8,4))
plt.scatter(range(len(y_test2)), y_test2, marker='o', s=70, label='Actual')
plt.scatter(range(len(y_pred2)), y_pred2, marker='x', s=70, label='Predicted')
plt.xlabel("Sample Number")
plt.ylabel("Class")
plt.title("Scenario 2: Actual vs Predicted")
plt.yticks([0,1,2], iris.target_names)
plt.legend()
plt.grid(True)
plt.show()

print("\nSCENARIO COMPARISON")
comparison = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1 Score"],
    "Scenario 1": [accuracy1, precision1, recall1, f11],
    "Scenario 2": [accuracy2, precision2, recall2, f12],
    "Difference": [
        accuracy1 - accuracy2,
        precision1 - precision2,
        recall1 - recall2,
        f11 - f12
    ]
})
print(comparison)

metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]
scenario1 = [accuracy1, precision1, recall1, f11]
scenario2 = [accuracy2, precision2, recall2, f12]

x = range(len(metrics))
width = 0.35
plt.figure(figsize=(8,5))
plt.bar([i-width/2 for i in x], scenario1, width=width, label="Scenario 1")

```

```
plt.bar([i+width/2 for i in x], scenario2, width=width, label="Scenario 2")
plt.xticks(x, metrics)
plt.ylabel("Score")
plt.title("Performance Metrics Comparison")
plt.ylim(0,1.1)
plt.grid(axis='y')
plt.legend()
plt.show()
```

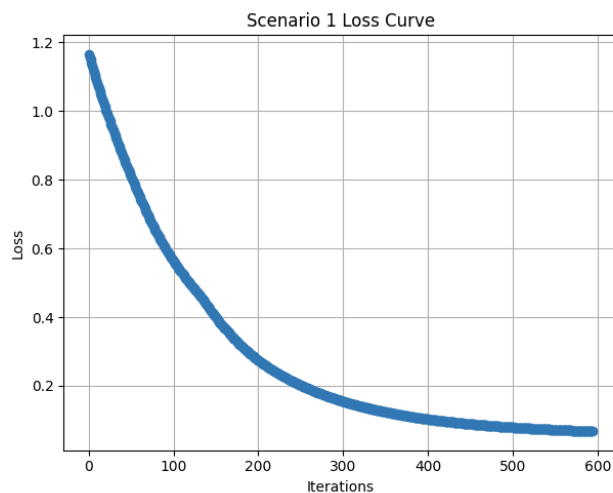
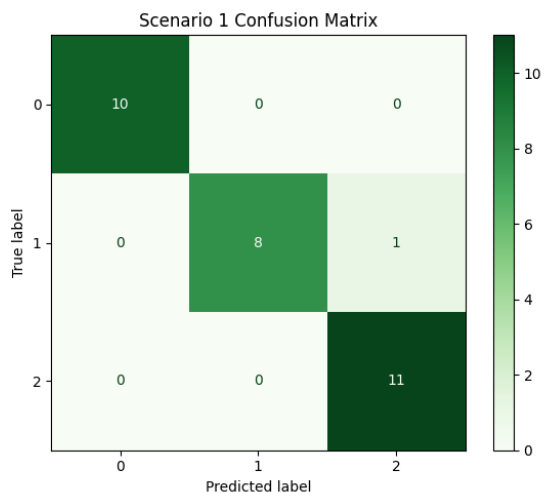
OUTPUT:

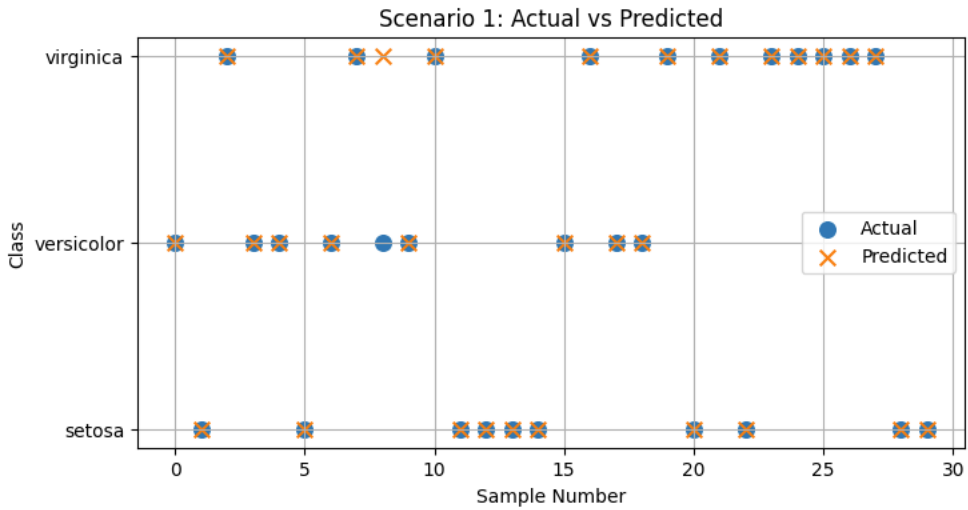
Scenario 1 Evaluation and Graphs:

SCENARIO 1 (80% Train - 20% Test)
 Accuracy : 0.9666666666666667
 Precision: 0.9694444444444444
 Recall : 0.9666666666666667
 F1 Score : 0.9664109121909632

Confusion Matrix
 [[10 0 0]
 [0 8 1]
 [0 0 11]]

Classification Report				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30





Scenario 2 Evaluation and Graphs:

```
*** SCENARIO 2 (90% Train - 10% Test)
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
```

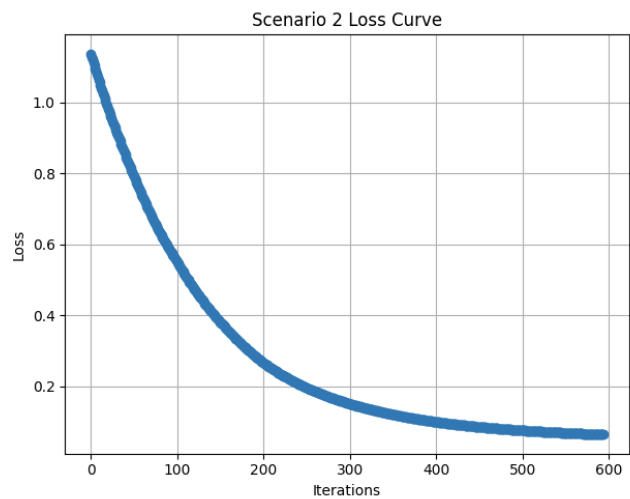
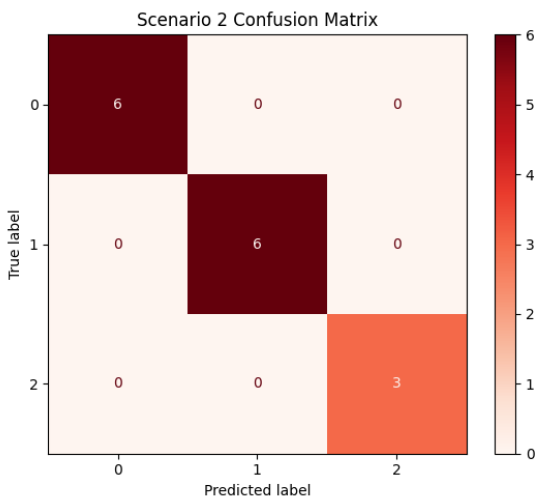
Confusion Matrix

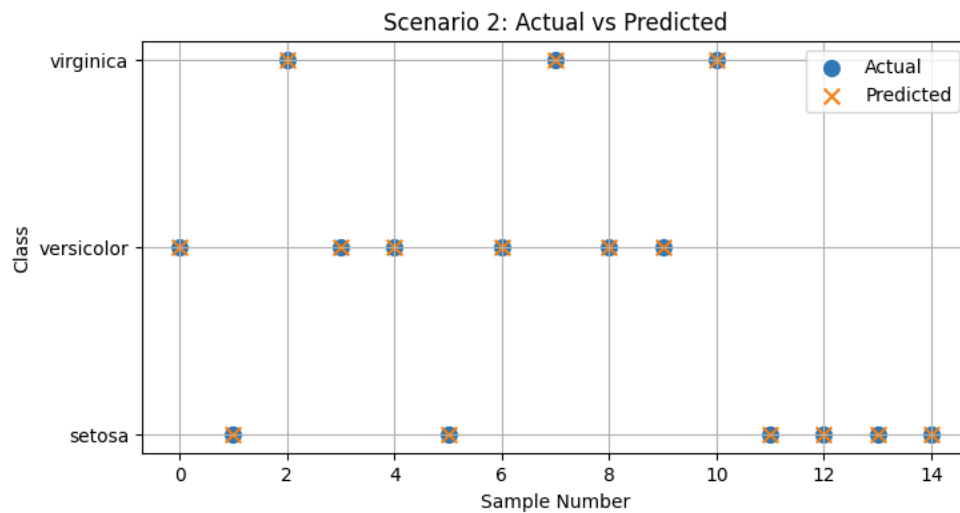
```
[[6 0 0]
 [0 6 0]
 [0 0 3]]
```

Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6
1	1.00	1.00	1.00	6
2	1.00	1.00	1.00	3
accuracy			1.00	15
macro avg	1.00	1.00	1.00	15
weighted avg	1.00	1.00	1.00	15

...





Scenario Comparisons:

SCENARIO COMPARISON				
	Metric	Scenario 1	Scenario 2	Difference
0	Accuracy	0.966667	1.0	-0.033333
1	Precision	0.969444	1.0	-0.030556
2	Recall	0.966667	1.0	-0.033333
3	F1 Score	0.966411	1.0	-0.033589

